



Artificial Intelligence course

6th Semester

Bachelor in Informatics and Computer Engineering

Lists

Nuno Leite

17 March 2022

Lists

- The *list* is a simple data structure widely used in non-numeric programming
- A list is a sequence of any number of items, such as `ann, tennis, tom, skiing`
- Such a list can be written in Prolog as:
`[ann, tennis, tom, skiing]`

Lists

- This is a friendlier syntax for standard Prolog *terms* that represent list structures
- All structured objects in Prolog are *trees*, and lists are no exception to this
- How can a list be represented as a structured Prolog object?
- We have to consider two cases:
 - A list is either *empty* or *non-empty*
 - In the first case, the list is simply written as the Prolog atom `[]`

Lists

- In the second case, the list can be viewed as consisting of two things:
 1. The first item, called the *head* of the list
 2. The remaining part of the list, called the *tail*
- For the example list `[ann, tennis, tom, skiing]` the head is `ann` and the tail is the list:
`[tennis, tom, skiing]`

Lists

- The head can be anything (any Prolog object – a constant, a variable, a tree, and also a list again)
- The tail has to be a list
- The head and the tail are then combined into a structure by the functor `'.'`:
`.(Head, Tail)`
- Remark: Recent versions of SWI-Prolog use the functor `'[]'` instead of `'.'`

Lists

- Since `Tail` is in turn a list, it is either empty or it has its own head and tail
- Therefore, to represent lists of any length, no additional principle for structuring objects is needed
- Our example list is represented in the standard Prolog notation as the term:

```
.(ann, .(tennis, .(tom, .(skiing, []))))
```

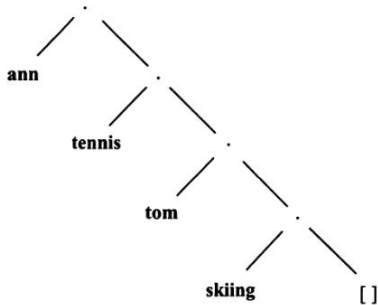


Figure 3.1 Tree representation of the list [**ann**, **tennis**, **tom**, **skiing**]

Lists

- Note that the empty list appears in this tree
- This is because the one but last tail is a single item list:
`[skiing]`
- This list has the empty list as its tail:
`[skiing] = .(skiing, [])`

Examples

```
1  ?- [a,b,c] = .(a, .(b, .(c, []))).
2  yes
3  ?- List1 = [a,b,c], List2 = .(a, .(b, .(c, []))).
4  List1 = [a,b,c]
5  List2 = [a,b,c]
6  ?- Hobbies1 = .(tennis, .(music, [])),
7     Hobbies2 = [skiing, food],
8     L = [ann, Hobbies1, tom, Hobbies2].
9
10 Hobbies1 = [tennis, music]
11 Hobbies2 = [skiing, food]
12 L = [ann, [tennis, music], tom, [skiing, food]]
```

Lists

- This example also confirms that the elements of a list can be objects of any kind
 - In particular they can also be lists
- It is often practical to treat the whole tail as a single object.

For example, let:

$L = [a, b, c]$

Then we could write:

$Tail = [b, c]$ and $L = .(a, Tail)$

The vertical bar notational extension

- Prolog provides another notational extension, the vertical bar, which separates the head and the tail:

$L = [a \mid Tail]$

- The vertical bar notation is in fact more general:
 - We can list any number of elements followed by ‘|’ and the list of remaining items
 - Thus alternative ways of writing the list $[a, b, c]$ are:
?- $L1 = [a, b, c]$, $L2 = [a \mid [b,c]]$,
 $L3 = [a, b \mid [c]]$, $L4 = [a, b, c \mid []]$.
 $L1 = L2$, $L2 = L3$, $L3 = L4$, $L4 = [a, b, c]$.

Some operations on lists – Membership

- Let us implement the *membership* relation as:

`member(X, L)`

where `X` is an object and `L` is a list. The goal `member(X, L)` is true if `X` occurs in `L`

- Examples:
 - `member(b, [a, b, c])` is true
 - `member(b, [a, [b, c]])` is not true
 - `member([b, c], [a, [b, c]])` is true

Membership

- The program for the membership relation can be based on the following observation:

X is a member of L if either:

1. X is the head of L, or
2. X is a member of the tail of L

This can be written in two clauses:

```
member(X, [X | Tail]).  
member(X, [Head | Tail]) :-  
    member(X, Tail).
```

Other uses of predicate *member*

Check item existence:

```
1  ?- member(b, [a, b, c]). % Check if b is in [a,b,c]
2  yes
```

Generate through backtracking all the members of a given list:

```
1  % Find such X that X is member of [a,b,c]
2  ?- member(X, [a, b, c]).
3  X = a;
4  X = b;
5  X = c;
6  no
```

Other uses of predicate *member*

Find lists that contain a given item, e.g. 'apple':

```
1  ?- member(apple, L).  
2  % Any list that has "apple" as the head  
3  L = [apple | _A];  
4  % First item is anything, second is "apple"  
5  L = [_A, apple | _B];  
6  L = [_A, _B, apple | _C];  
7  ...
```

Other uses of predicate *member*

Another idea is: find lists that contain a, b and c:

```
1  ?- member(a, L), member(b, L), member(c, L).  
2  L = [a, b, c | _A];  
3  L = [a, b, _A, c | _B];  
4  L = [a, b, _A, _B, c | _C];  
5  L = [a, b, _A, _B, _C, c | _D];  
6  ...
```


Other uses of predicate *member*

- Prolog never gets to alternative permutations of *a*, *b* and *c* because of the order in which the solutions are generated
 - First, *a* is put in the list as the list's head,
 - then *b* as the second element of the list,
 - and then (potentially infinitely many!) lists that contain *c* are generated

Other uses of predicate *member*

Generating permutations:

```
1  % L is any list with exactly three elements
2  ?- L = [_, _, _],
3      member(a, L), member(b, L), member(c, L).
4  L = [a, b, c];
5  L = [a, c, b];
6  L = [b, a, c];
7  L = [c, a, b];
8  L = [b, c, a];
9  L = [c, b, a];
10 no
```